

Building a Claude Code Plugin — Interactive Guide

A general-purpose walkthrough for building any Claude Code plugin: what a plugin is, what pieces it can contain, how to structure the files, how to register and install it locally, and how to iterate on it.

Every action is a `- [ ]` checkbox. Every large code sample lives inside a `<details><summary>▶ Show code</summary>` block. Tick as you go; expand only what you need.

0. What a Claude Code plugin is

A **plugin** is a folder that Claude Code loads at startup. It can contribute any combination of:

Capability	Where it lives	What it does
Slash commands	<code>commands/*.md</code>	Adds <code>/plugin-name:command-name</code> to the picker.
Hooks	<code>hooks/hooks.json</code>	Runs shell commands on lifecycle events (SessionStart, PreToolUse, Stop, ...).
Agents	<code>agents/*.md</code>	Adds specialised subagents callable via the Agent tool.
Skills	<code>skills/<name>/SKILL.md</code>	Reusable procedures Claude can invoke via the Skill tool.
MCP servers	<code>.mcp.json</code>	Adds tools/resources served by a Model Context Protocol server.
Statusline	<code>statusline/*</code>	Custom statusline renderer.

A plugin must contain **at least one** of these. It always includes a plugin **manifest**.

Plugins are distributed through **marketplaces** — a JSON descriptor that lists one or more plugins. You can point Claude Code at:

- A **local folder** (the dev workflow — this guide focuses on it).
- A **git repository** on GitHub / GitLab / any git host.
- A **subdirectory** of a git repository.

1. Prerequisites

- ☐ **Claude Code** installed (CLI, IDE, or desktop). Recent version.
 - ☐ A shell (bash, zsh, PowerShell) — needed by hooks and slash commands that shell out.
 - ☐ Any runtime your plugin needs (Node, Python, Deno, ...). Prefer **Node** if you need cross-platform JSON handling because it ships with `fs` and `JSON` and no extra dependencies.
 - ☐ A working directory where you'll develop the plugin. The `.claude/settings.json` at that directory's root controls which plugins are enabled for sessions started there.
-

2. Decide what your plugin will do

Answer these before writing any files. Two minutes now saves ten later.

- ☐ **Trigger surface.** Slash command? Hook? Both? Something else (skill, agent)?
- ☐ **Inputs and outputs.** What does the user pass in? What appears in the conversation as output?
- ☐ **Side effects.** Does it read/write files? Call APIs? Update user state across sessions?
- ☐ **Failure modes.** What happens if a dependency is missing, network is down, or state is corrupt? Fail loudly with a clear message, or silently degrade?
- ☐ **State.** Does it need persistent memory across invocations? If yes, where — `~/.claude/<plugin-name>-state.json` is a common convention.

Write your answers down. They become the design section of a spec if the plugin gets non-trivial.

3. Scaffold the folder

Pick a **plugin name** (lower-case, kebab-case, no spaces). Everywhere below, replace `my-plugin` with your chosen name.

Create this skeleton inside your working directory:

```
plugins/
├── .claude-plugin/
│   └── marketplace.json      ← lists your plugins
├── my-plugin/
│   └── .claude-plugin/
│       └── plugin.json      ← plugin manifest
```

Everything else (commands, hooks, scripts, tests, data) is optional and added as needed.

- ☐ **Step 3.1** — Create the **marketplace descriptor** at `plugins/.claude-plugin/marketplace.json`.

▼ ► Show code

```
{
  "name": "local-plugins",
  "owner": { "name": "your-name-or-org" },
  "plugins": [
    {
      "name": "my-plugin",
      "source": "./my-plugin",
      "description": "One-line summary of what my-plugin does"
    }
  ]
}
```

Notes:

- `name` is the marketplace name; users will refer to your plugin as `my-plugin@local-plugins`.
- `source` is a path **relative to the marketplace.json file**.
- Add more entries to the `plugins` array to publish multiple plugins from one marketplace.

- ☐ **Step 3.2** — Create the **plugin manifest** at `plugins/my-plugin/.claude-plugin/plugin.json`.

▼ ► Show code

```
{
  "name": "my-plugin",
  "version": "0.1.0",
  "description": "One-line summary of what my-plugin does"
}
```

`name` here must match the `name` in `marketplace.json`. Bump `version` when you release changes.

- ☐ **Step 3.3** — Validate both JSON files parse.

```
node -e "JSON.parse(require('fs').readFileSync('plugins/.claude-plugin/marketplace.json', 'utf8'))"
node -e "JSON.parse(require('fs').readFileSync('plugins/my-plugin/.claude-plugin/plugin.json', 'utf8'))"
```

4. Add a slash command

Slash commands appear in the picker as `/<plugin-name>:<command-name>`. Each command is a single markdown file in `commands/`.

- ❑ **Step 4.1** — Create `plugins/my-plugin/commands/hello.md`.

▼ ▶ Show code

```
---
description: One-line summary shown in the /command picker
allowed-tools: Bash
---

!node "${CLAUDE_PLUGIN_ROOT}/scripts/hello.js"
```

Anatomy:

- **Frontmatter** — YAML at the top declares the command's metadata.
 - `description` — required. Shown in the picker.
 - `allowed-tools` — required if the body uses tools. `Bash` lets the body run shell commands via the `!` prefix.
- **Body** — instructions Claude follows when the command runs. Lines starting with `!` are executed as shell commands; their stdout becomes context for the model. Any regular markdown becomes an instruction prompt.
- `${CLAUDE_PLUGIN_ROOT}` — expanded by Claude Code to the absolute path of the installed plugin folder. Always use it. Never hardcode absolute paths.

- ❑ **Step 4.2** — Create the script the command shells out to. Keeping logic in a script (not inline in the markdown) makes it testable and reusable across hook + command.

▼ ▶ Show code — `plugins/my-plugin/scripts/hello.js`

```
#!/usr/bin/env node
"use strict";

// Everything a plugin script writes to stdout becomes context Claude sees.
// Keep it short, meaningful, and self-contained.

const who = process.argv[2] || process.env.USER || "friend";
process.stdout.write(`Hello, ${who}! It is ${new Date().toISOString()}\n`);
```

Slash command anatomy — the three body styles:

Style	Body example	What Claude does
Instruction only	Summarise the last commit.	Treats the body as a user prompt.
Shell command via !	!git log -1 --stat	Runs the command; stdout becomes context; Claude responds based on that context.
Instruction + shell context	Explain this diff:\n\n!git diff HEAD~1	Runs the shell command, then acts on its output following the instruction.

5. Add a hook

Hooks run shell commands on **lifecycle events**. Each hook writes to stdout, which Claude Code injects as additional context for the event.

Common events:

Event	When it fires	Typical use
SessionStart	A new conversation begins in a directory with the plugin.	Greetings, project status, priming context.
PreToolUse	Before Claude calls a specific tool.	Gate destructive commands; enrich tool input.
PostToolUse	After a tool call finishes.	Log, audit, or react to tool results.
Stop	The assistant finishes responding.	Cleanup, notifications, telemetry.
UserPromptSubmit	User submits a prompt.	Rewrite / annotate / block the prompt before Claude sees it.

- ☐ **Step 5.1** — Create `plugins/my-plugin/hooks/hooks.json`.

▼ Show code

```
{
  "hooks": {
    "SessionStart": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "node \"${CLAUDE_PLUGIN_ROOT}/scripts/hello.js\""
          }
        ]
      }
    ]
  }
}
```

You can attach multiple commands to the same event, or use different events. The outer array is a list of "matchers" (each with its own list of hook commands); the inner array is the commands to run for that matcher.

- ☐ **Step 5.2** — Validate the hook JSON:

```
node -e "JSON.parse(require('fs').readFileSync('plugins/my-plugin/hooks/hooks.json', 'utf8'))"
```

6. (Optional) Add an agent

Agents are specialised subagents Claude invokes via the Agent tool. They have their own tool allowlist, own model, own system prompt.

- ☐ **Step 6.1** — Create `plugins/my-plugin/agents/reviewer.md`.

▼ ► Show code

```
---
name: reviewer
description: Reviews a diff for style issues and typos. Invoke when finalising a PR.
tools: Read, Grep, Glob
---

You are a focused reviewer. Look at the given diff and return a bulleted list of
issues, ordered by severity. Be terse. If there are no issues, say so in one line.
```

Agents show up in the Agent tool's `subagent_type` list as `my-plugin:reviewer`.

7. (Optional) Add a skill

Skills package a reusable procedure — a "here's how we do X" — that Claude can pull into context via the Skill tool.

- ☐ **Step 7.1** — Create `plugins/my-plugin/skills/refactor-notes/SKILL.md`.

▼ ► Show code

```
---
name: refactor-notes
description: Use when the user asks to refactor code and wants change notes in a consis
---

# Refactor Notes

When refactoring, always produce a short note capturing:

1. Motivation — one sentence.
2. Change summary — bullet list of touched files.
3. Behavioural impact — anything an integrator needs to know.
4. Tests — what you ran, what passed.

Keep the note under 200 words.
```

Skills are shown in the assistant's available-skills list as `my-plugin:refactor-notes` .

8. (Optional) Add an MCP server

If your plugin exposes tools via the Model Context Protocol, declare a server in `.mcp.json` at the plugin root.

▼ ► Show code — `plugins/my-plugin/.mcp.json`

```
{
  "mcpServers": {
    "my-plugin-tools": {
      "command": "node",
      "args": ["${CLAUDE_PLUGIN_ROOT}/mcp-server/index.js"]
    }
  }
}
```

Building the MCP server itself is beyond this guide — see the MCP spec.

9. Register the local marketplace (the important gotcha)

You have the files on disk. Now you have to tell Claude Code about them.

- ☐ **Step 9.1** — Enable the plugin in `.claude/settings.json`:

▼ ► Show code

```
{
  "enabledPlugins": {
    "my-plugin@local-plugins": true
  }
}
```

- ☐ **Step 9.2** — From inside Claude Code (not the shell), register the marketplace:

```
/plugin marketplace add ./plugins
```

Gotcha: an `extraKnownMarketplaces` key inside `settings.json` looks like it should work for local marketplaces, but in practice it doesn't. The `/plugin marketplace add` command is the correct dev flow — it writes into `~/.claude/plugins/known_marketplaces.json` where Claude Code actually looks.

- ☐ **Step 9.3** — Install your plugin from that marketplace:

```
/plugin install my-plugin@local-plugins
```

- ☐ **Step 9.4** — Reload plugins:

```
/reload-plugins
```

Look at the reload summary: `Reloaded: N plugins · N skills · N agents · N hooks · ...`. The numbers you contribute (commands, hooks, agents, skills, MCP servers) should be reflected in the totals.

10. Verify end to end

- ☐ **Step 10.1** — Invoke your command:

```
/my-plugin:hello
```

Expected: your script's output appears.

- ☐ **Step 10.2** — Start a fresh session in this directory (exit Claude Code and reopen). If you added a `SessionStart` hook, its output should appear at the top of your first response.
 - ☐ **Step 10.3** — Confirm no crashes in the reload summary and no permission errors in the terminal.
-

11. Debug playbook

▼ ► My slash command doesn't appear in the picker

- Did you run `/plugin marketplace add ./plugins` **from Claude Code**? Editing `settings.json` alone does not register a local marketplace.
- Did you run `/plugin install my-plugin@local-plugins` and then `/reload-plugins` ?
- Check the command's frontmatter — `description` must be present. Missing frontmatter can silently exclude it.
- The command name is namespaced: `/my-plugin:hello` , not just `/hello` .

▼ ► My hook doesn't fire

- Validate `hooks.json` :

```
node -e "JSON.parse(require('fs').readFileSync('plugins/my-plugin/hooks/hooks.json', 'utf8'))"
```

- Confirm the reload summary shows the expected hook count.
- Ensure the interpreter (`node` , `python` , `pwsh` , ...) is on the PATH used by Claude Code.
- Test the underlying script directly outside Claude Code — the hook does nothing more than run the command.

▼ ► Unicode / non-ASCII output looks garbled

- Save all files as UTF-8 **without** BOM.
- Windows terminals may need `chcp 65001` for UTF-8 in `cmd.exe` . Windows Terminal, VS Code integrated terminal, and Git Bash default to UTF-8.
- If your script writes to a file, pass an explicit `utf8` encoding.

▼ ► Permission prompts on every invocation

- Add read-only or safe commands to `.claude/settings.json` under a `permissions.allow` list.
- Or use the `/fewer-permission-prompts` skill to auto-collect commonly-used commands from your transcripts.

▼ ► How do I reset state?

Delete the file your plugin writes state to (typically `~/.claude/<plugin-name>-state.json`). Recreated on next invocation.

▼ ► Changes to data files aren't reflected

If your script reads a data file on every invocation (recommended), edits take effect immediately — no reload needed. If your script caches the data in memory (e.g., a long-running MCP server), you must restart the process.

12. Publishing beyond local

Once the plugin works locally, share it by pointing others at your marketplace over git.

- ☐ Push the `plugins/` folder to a git repository.
- ☐ Users register it with:

```
/plugin marketplace add github:your-user/your-repo
```

or

```
/plugin marketplace add https://github.com/your-user/your-repo
```

- ☐ They install and enable normally.
- ☐ For versioning, tag releases in git; the marketplace can be pinned to a `ref` or `sha` .

If you want to be listed in the official `claude-plugins-official` marketplace, follow that repo's contribution guidelines. Otherwise, hosting your own git-based marketplace is the recommended path.

13. Reusable patterns

- **The four-file skeleton** (`plugin.json` , `commands/` , `hooks/` , `scripts/`) covers most plugins. Add `agents/` , `skills/` , `.mcp.json` only when you need them.
 - **Keep logic out of markdown.** Put the real work in scripts under `scripts/` so you can unit-test them. Commands and hooks just shell out.
 - **Node over bash on Windows.** Cross-platform JSON, Unicode-safe, ships everywhere Claude Code runs. Use bash only if you're sure about the target environment.
 - `${CLAUDE_PLUGIN_ROOT}` **everywhere.** In `hooks.json` , command bodies, and any script paths embedded in metadata. Never hardcode absolute paths.
 - **User-scope state under** `~/.claude/<plugin-name>-state.json` . One namespaced file per plugin; JSON is easy to inspect and reset.
 - **Fail loudly at boundaries.** If a required binary is missing, print a clear error to stderr and exit non-zero. Don't silently degrade — the user should know why nothing happened.
 - **Small, focused plugins.** One plugin doing one job well is easier to maintain than a monolith with five modes. Ship them as separate entries in your marketplace if you have multiple.
-

14. Checklist to publish v0.1.0

Before you tag a release:

- ☐ `plugin.json` name, version, description accurate.
- ☐ `marketplace.json` includes the plugin.
- ☐ Every `command` markdown has a clear `description` and any needed `allowed-tools` .
- ☐ `hooks/hooks.json` (if any) validates as JSON and the referenced scripts exist.
- ☐ Any scripts use `${CLAUDE_PLUGIN_ROOT}` , not absolute paths.
- ☐ Scripts run standalone (test them outside Claude Code first).
- ☐ `README.md` at the plugin root explains what it does, install steps, and any dependencies.
- ☐ Tag the git commit; note the ref in the marketplace entry if you plan to pin.

Happy building. 🛠️